Lesson plan · for the teacher

Your first model in Python

Optimization · Level 2 · Lesson 4

Time
40 min

Materials
A phone with Darsi (one is enough), and a projector or a small group around it; Board and chalk; Paper copies of the garden model for classes without devices

Learning goals

- Write the garden model in Python with OR-Tools.
- Read what the solver says: OPTIMAL, FEASIBLE, INFEASIBLE.
- Check the solver's answer and find a missing rule.

Activity: Model it together

1. Warm-up (5 min): Show the garden graph from last lesson. Ask: how would we draw the timetable, with hundreds of lessons? Explain that a solver can search for us.
2. Rules (10 min): Choose a problem, such as sharing 30 textbooks between three classes. Write in plain words: what we decide (books for each class), the rules (30 in all, at least 8 each) and the goal (more for bigger classes, or as equal as possible).
3. Code (10 min): With one phone on a projector, or the class around you, open the modeller lab and type the model line by line, pointing to the rule on the board each line comes from.
4. Run and check (10 min): Run it. Check the answer against every rule on the board. Ask whether the answer is fair, and what the class would change.
5. Wrap-up (5 min): Add the rule the class suggests, run it again and compare. Agree: the solver only knows the rules we write.

Practise the model once before class. Python downloads the first time (about 0.3 MB): run it once while online. No phone? Pupils write the model on paper, and you run it later.

Discussion

- Which line of the program matches which rule on the board?
- What did the solver get "right" that was still wrong for people?
- What does INFEASIBLE tell us, and who decides what changes?

Exit ticket

- Write one constraint as a model.add(...) line, and say which rule it stands for.

Try it at school tomorrow
Model a small class problem together, such as sharing 30 textbooks between three classes so that every class gets at least 8. Write the variables and rules on the board in plain words, then type them into the modeller lab and run it in front of the class. Ask: what rule did we forget?

Practice questions · for you

Your first model in Python

Optimization · Level 2 · Lesson 4

1 **Mr Joseph's timetable model prints INFEASIBLE. What does that tell him?**

◯ Run it again until it works ◯ The solver is broken ◯ No timetable keeps all the rules: one must change

2 **Only tomatoes this year: each bed needs 2 cans of water a day, and there are 14. Put the lines in the order a model is built: the model, the variable, the limit, the goal, then the solver, solve and print.**

Number the lines in order (1, 2, 3...). Cross out any line that doesn't belong.

☐ `model.add(2 * beds <= 14)`

☐ `beds = model.new_int_var(0, 10, "beds")`

☐ `solver.solve(model)`

☐ `solver = cp_model.CpSolver()`

☐ `from ortools.sat.python import cp_model`

☐ `print(solver.value(beds))`

☐ `model.add(2 * beds >= 14)`

☐ `model = cp_model.CpModel()`

☐ `print(beds)`

☐ `model.maximize(30 * beds)`

3 **The same CP-SAT program runs unchanged on Google's real OR-Tools on a computer or in Colab.**

◯ True ◯ False

4 **Ms Salma's garden model says: plant nothing, profit 0. One line is wrong. Which one?**

Circle the line with the mistake. Write it correctly:

```
1  model = cp_model.CpModel()
2  x = model.new_int_var(0, 10, "tomato")
3  y = model.new_int_var(0, 10, "beans")
4  model.add(x + y <= 8)
5  model.add(2 * x + y <= 14)
6  model.add(x + 3 * y <= 18)
7  model.minimize(30 * x + 40 * y)
```

5 **The solver says OPTIMAL. What should you still do before you use its plan? Choose all that apply.**

☐ Change one number and see if the answer moves sensibly ☐ Run it again to see if it changes its mind

☐ Nothing: OPTIMAL means the plan is right for people ☐ Ask the people who will live with the plan

☐ Put the answer back into every real limit

✎ **Which small problem at your school could your pupils model in Python? What rule would they be most likely to forget, and who would notice?**

Check your answers

Your first model in Python

Optimization · Level 2 · Lesson 4

1 **No timetable keeps all the rules: one must change**

INFEASIBLE is not a crash: the solver has proved that no timetable keeps every rule, and running it again gives the same answer.

2 **Top to bottom: 4, 3, 7, 6, 1, 8, X, 2, X, 5**

```
from ortools.sat.python import cp_model
model = cp_model.CpModel()
beds = model.new_int_var(0, 10, "beds")
model.add(2 * beds <= 14)
model.maximize(30 * beds)
solver = cp_model.CpSolver()
solver.solve(model)
print(solver.value(beds))
```

Each line uses what the lines before it made: the variable needs the model, the limit and goal need the variable, solving needs the solver and the model, and the value can only be read after solving.

3 **True**

Darsi's small OR-Tools is shaped like Google's, so the same program runs on both.

4 **Line 7: `model.maximize(30 * x + 40 * y)`**

minimize asks for the smallest profit, and planting nothing earns the least: 0.

5 **Put the answer back into every real limit; Ask the people who will live with the plan; Change one number and see if the answer moves sensibly**

OPTIMAL is proved for the model as written, not for the real problem.